UMAT CAMPUS COMPLAINT SYSTEM

Backend Architecture & MySQL Database Specification

Document Version: 1.0.0
Status: Draft Architecture Spec
Author: Antigravity AI Architect

Target Database System: MySQL 8.0+
Target Server Stack: Node.js (Express) or Python (Flask/Django)
Date Generated: July 6, 2026

1. Introduction & Overview

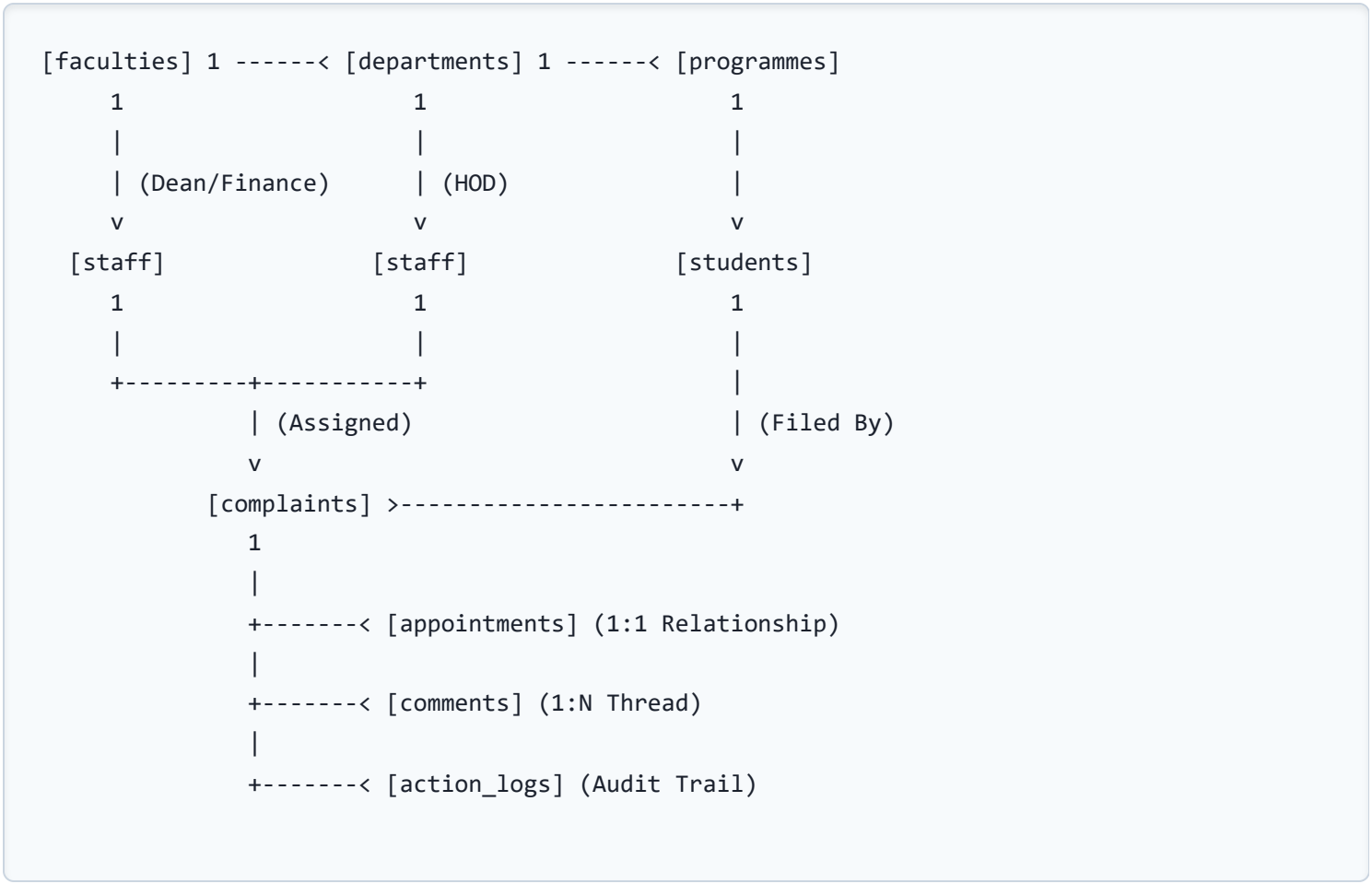
The **UMaT Campus Complaint System** is designed to manage and resolve student complaints and grievances. The backend is designed as a secure, transactional, stateless RESTful API that handles user authentication (students and staff/administrators), grievance filing, automated routing rules to specific university authorities (Deans, Finance, IT Directorate), internal commenting/messaging, counselor appointments, and operational logs.

The backend will connect to a relational **MySQL Database**, enforcing high integrity through relational constraints (foreign keys), schemas optimized for index lookup speed, and transactional audit trails.

2. MySQL Database Schema

The MySQL schema contains tables representing faculties, departments, academic programmes, student profiles, administrative staff, tickets/complaints, workflow action records, messaging threads, and counselor appointments.

Entity-Relationship Mapping Diagram



MySQL Table Schemas

2.1 Faculties (`faculties`)

Field	Type	Constraints	Description
faculty_key	VARCHAR(10)	PRIMARY KEY	Short key (e.g. FMMT, FCMS, FGES)
name	VARCHAR(150)	NOT NULL	Full name of the faculty

2.2 Departments (`departments`)

Field	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for departments
name	VARCHAR(150)	NOT NULL, UNIQUE	Full name of the department

2.3 Programmes (`programmes`)

Field	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for academic programs
name	VARCHAR(150)	NOT NULL	Name of degree (e.g. BSc Computer Science)
department_id	INT	FOREIGN KEY	References `departments(id)`
faculty_key	VARCHAR(10)	FOREIGN KEY	References `faculties(faculty_key)`

2.4 Administrative Staff / Admins (`staff`)

Field	Type	Constraints	Description
staff_id	VARCHAR(20)	PRIMARY KEY	University Staff ID (e.g. DEC-102)
name	VARCHAR(100)	NOT NULL	Full name of staff member
email	VARCHAR(100)	NOT NULL, UNIQUE	Official university email
password_hash	VARCHAR(255)	NOT NULL	BCrypt hashed password
type	ENUM	NOT NULL	'Dean', 'Finance', 'IT', 'HOD', 'SuperAdmin'
faculty_key	VARCHAR(10)	NULL, FOREIGN KEY	References `faculties` (for Deans/Finance)
department_id	INT	NULL, FOREIGN KEY	References `departments` (for HODs)
portfolio	VARCHAR(150)	NOT NULL	Job designation description

2.5 Students (`students`)

Field	Type	Constraints	Description
index_number	VARCHAR(15)	PRIMARY KEY	Student Registration Index (e.g. 9021020)
name	VARCHAR(100)	NOT NULL	Full name of student
email	VARCHAR(100)	NOT NULL, UNIQUE	Official student email
phone	VARCHAR(20)	NOT NULL	Active telephone contact
password_hash	VARCHAR(255)	NOT NULL	BCrypt hashed password
level	VARCHAR(10)	NOT NULL	Year status (100, 200, 300, 400, etc.)
programme_id	INT	FOREIGN KEY	References `programmes(id)`

2.6 Complaints / Grievance Tickets (`complaints`)

Field	Type	Constraints	Description
id	VARCHAR(25)	PRIMARY KEY	Ticket Reference Code (e.g. UMAT-2026-X12Y)
student_index	VARCHAR(15)	FOREIGN KEY	References `students(index_number)`
subject	VARCHAR(150)	NOT NULL	Brief grievance title
category_id	VARCHAR(20)	NOT NULL	'academic', 'finance', 'ict', 'harassment', 'other'
urgency	ENUM	NOT NULL	'Low', 'Medium', 'High', 'Urgent'
description	TEXT	NOT NULL	Detailed explanation of issue
status	ENUM	NOT NULL	'Submitted', 'Under Review', 'In Progress', 'Resolved', 'Rejected'
assigned_staff_id	VARCHAR(20)	NULL, FOREIGN KEY	References `staff(staff_id)`
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time filed
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE	Last updated

2.7 Activity & Action Logs (`action_logs`)

Field	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for logs
complaint_id	VARCHAR(25)	FOREIGN KEY	References `complaints(id)` ON DELETE CASCADE
operator_name	VARCHAR(100)	NOT NULL	Name of staff making modification
action_type	VARCHAR(50)	NOT NULL	e.g. status_changed, appointment_booked
details	TEXT	NOT NULL	Detailed change logs
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time event logged

2.8 Counselor Appointments (`appointments`)

Field	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for meetings
complaint_id	VARCHAR(25)	FOREIGN KEY, UNIQUE	References `complaints(id)` (1-to-1 Mapping)
type	ENUM	NOT NULL	'Counselor Session', 'In-Person Dean Meeting', 'Finance Office Visit', 'IT Helpdesk Visit'
date_time	DATETIME	NOT NULL	Scheduled date and hour of appointment
venue	VARCHAR(150)	NOT NULL	Room / Office location details
checklist	TEXT	NOT NULL	JSON formatted requirements checklist array
counselor_name	VARCHAR(100)	NULL	Counseling officer assigned
status	ENUM	NOT NULL	'Scheduled', 'Completed', 'Cancelled'

2.9 Thread Comments (`comments`)

Field	Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique message id
complaint_id	VARCHAR(25)	FOREIGN KEY	References `complaints(id)` ON DELETE CASCADE
sender_type	ENUM	NOT NULL	'student', 'staff'
sender_name	VARCHAR(100)	NOT NULL	Display name of sender
message	TEXT	NOT NULL	Content body of text
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Sending time

3. MySQL Database Initial Setup SQL Script

The following raw SQL script initializes the tables, relationships, and basic indexes needed for performance optimizations:

```
CREATE DATABASE IF NOT EXISTS umat_complaints_db;
USE umat_complaints_db;

-- 1. Faculties
CREATE TABLE faculties (
    faculty_key VARCHAR(10) PRIMARY KEY,
    name VARCHAR(150) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. Departments
CREATE TABLE departments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. Programmes
CREATE TABLE programmes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    department_id INT NOT NULL,
    faculty_key VARCHAR(10) NOT NULL,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON UPDATE CASCADE,
    FOREIGN KEY (faculty_key) REFERENCES faculties(faculty_key) ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. Administrative Staff
CREATE TABLE staff (
    staff_id VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    type ENUM('Dean', 'Finance', 'IT', 'HOD', 'SuperAdmin') NOT NULL,
    faculty_key VARCHAR(10) NULL,
    department_id INT NULL,
    portfolio VARCHAR(150) NOT NULL,
    FOREIGN KEY (faculty_key) REFERENCES faculties(faculty_key) ON UPDATE CASCADE,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. Students
CREATE TABLE students (
    index_number VARCHAR(15) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
```

```
    phone VARCHAR(20) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    level VARCHAR(10) NOT NULL,
    programme_id INT NOT NULL,
    FOREIGN KEY (programme_id) REFERENCES programmes(id) ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 6. Complaints (Tickets)
CREATE TABLE complaints (
    id VARCHAR(25) PRIMARY KEY,
    student_index VARCHAR(15) NOT NULL,
    subject VARCHAR(150) NOT NULL,
    category_id VARCHAR(20) NOT NULL,
    urgency ENUM('Low', 'Medium', 'High', 'Urgent') NOT NULL,
    description TEXT NOT NULL,
    status ENUM('Submitted', 'Under Review', 'In Progress', 'Resolved', 'Rejected') DEFAULT 'Subr
    assigned_staff_id VARCHAR(20) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (student_index) REFERENCES students(index_number) ON UPDATE CASCADE,
    FOREIGN KEY (assigned_staff_id) REFERENCES staff(staff_id) ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 7. Action / Audit Logs
CREATE TABLE action_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    complaint_id VARCHAR(25) NOT NULL,
    operator_name VARCHAR(100) NOT NULL,
    action_type VARCHAR(50) NOT NULL,
    details TEXT NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (complaint_id) REFERENCES complaints(id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 8. Counselor/Meeting Appointments
CREATE TABLE appointments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    complaint_id VARCHAR(25) NOT NULL UNIQUE,
    type ENUM('Counselor Session', 'In-Person Dean Meeting', 'Finance Office Visit', 'IT Helpdesl
    date_time DATETIME NOT NULL,
    venue VARCHAR(150) NOT NULL,
    checklist TEXT NOT NULL, -- JSON string representation
    counselor_name VARCHAR(100) NULL,
    status ENUM('Scheduled', 'Completed', 'Cancelled') DEFAULT 'Scheduled',
    FOREIGN KEY (complaint_id) REFERENCES complaints(id) ON DELETE CASCADE ON UPDATE CASCADE
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 9. Thread Comments
CREATE TABLE comments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    complaint_id VARCHAR(25) NOT NULL,
    sender_type ENUM('student', 'staff') NOT NULL,
    sender_name VARCHAR(100) NOT NULL,
    message TEXT NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (complaint_id) REFERENCES complaints(id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- INDEX OPTIMISATIONS FOR LOOKUP PERFORMANCE
CREATE INDEX idx_complaints_student ON complaints(student_index);
CREATE INDEX idx_complaints_assigned ON complaints(assigned_staff_id);
CREATE INDEX idx_complaints_status ON complaints(status);
CREATE INDEX idx_comments_complaint ON comments(complaint_id);
```

4. Backend RESTful API Specification

The backend service exposes JSON REST endpoints. All authenticated requests (except login) must transmit a JWT token in the `Authorization: Bearer <TOKEN>` header.

4.1 Authentication

POST **`/api/auth/student/login`**

Description: Standard login endpoint for student credentials verification.

Request Body:

```
{
  "index_number": "9021020",
  "password": "password123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsIn...",
  "student": {
    "index_number": "9021020",
    "name": "Adjoa Mansah",
    "email": "amansah020@umat.edu.gh",
    "level": "300",
    "programme": "BSc Computer Science and Engineering"
  }
}
```

POST `/api/auth/staff/login`

Description: Verification endpoint for official administrator/staff credentials.

Request Body:

```
{
  "staff_id": "DEC-102",
  "password": "password123"
}
```

4.2 Complaints Management

POST `/api/complaints`

Description: Creates a new grievance ticket. The backend automatically determines the initial routing (to specific Faculty Deans or IT Directorate) based on the database routing config map.

Request Headers: `Authorization: Bearer <Token>`

Request Body:

```
{
  "subject": "Missing Grade for CSE 352",
  "category_id": "academic",
  "urgency": "High",
  "description": "My grades for Database Systems have not been posted on my dashboard..."
}
```

GET `/api/complaints/student/:index`

Description: Fetches all complaints filed by a particular student index.

GET `/api/complaints/staff/:staffId`

Description: Admin-only. Retrieves all grievances routed to a staff member's jurisdiction (e.g. Dean of FCMS retrieves FCMS students' academic disputes).

PUT `/api/complaints/:id/status`

Description: Admin-only. Modifies status of a grievance ticket and registers an action log entry.

Request Body:

```
{
  "status": "In Progress",
  "operator_name": "Prof. Anthony Simons",
  "reason": "Moving ticket to active investigation with registry offices."
}
```

4.3 Comments & Chat Threads

GET `/api/complaints/:id/comments`

Description: Retrieves the message history logged in the thread under a complaint ticket.

POST `/api/complaints/:id/comments`

Description: Sends a comment on a ticket thread.

Request Body:

```
{
  "sender_type": "student",
  "sender_name": "Adjoa Mansah",
  "message": "Thank you, I will bring the printed slip on Friday."
}
```

4.4 Appointments Management

POST `/api/appointments`

Description: Admin-only. Books a counselor or dean appointment and sets up action ledger items.

Request Body:

```
{
  "complaint_id": "UMAT-2026-X12Y",
  "type": "In-Person Dean Meeting",
  "date_time": "2026-07-10 10:00:00",
  "venue": "Dean's Office, Room 4",
  "counselor_name": "Prof. Anthony Simons",
  "checklist": ["Bring student ID card", "Bring printed payment receipt history"]
}
```

5. Security, Validation & Business Logic Rules

To ensure transaction consistency and database security, the following backend principles must be implemented in code:

- **Password Encryption:** Use strong one-way hashing functions (such as BCrypt with 10 salt rounds or Argon2id) to encrypt student and staff passwords before database persistence.
- **Role-Based Access Control (RBAC):** Standard students must only be allowed to access their own filed tickets, action logs, and thread conversations. Admin endpoints (e.g. updating statuses, listing department tickets, scheduling counselor dates) must reject standard student JWT claims.
- **Transaction Enforcements:** When creating counselor appointments or updating status codes, perform the actions within a SQL transaction block so that the ticket status update, the log entry creation, and the appointment assignment succeed or fail as a single atomic write.
- **Sanitization:** Input fields (especially VARCHAR descriptions, text message comments, and subjects) must be sanitized and parameterised to prevent SQL Injection vectors (which is natively supported by modern ORMs like Sequelize, Knex, or raw SQL queries with placeholders `?`).